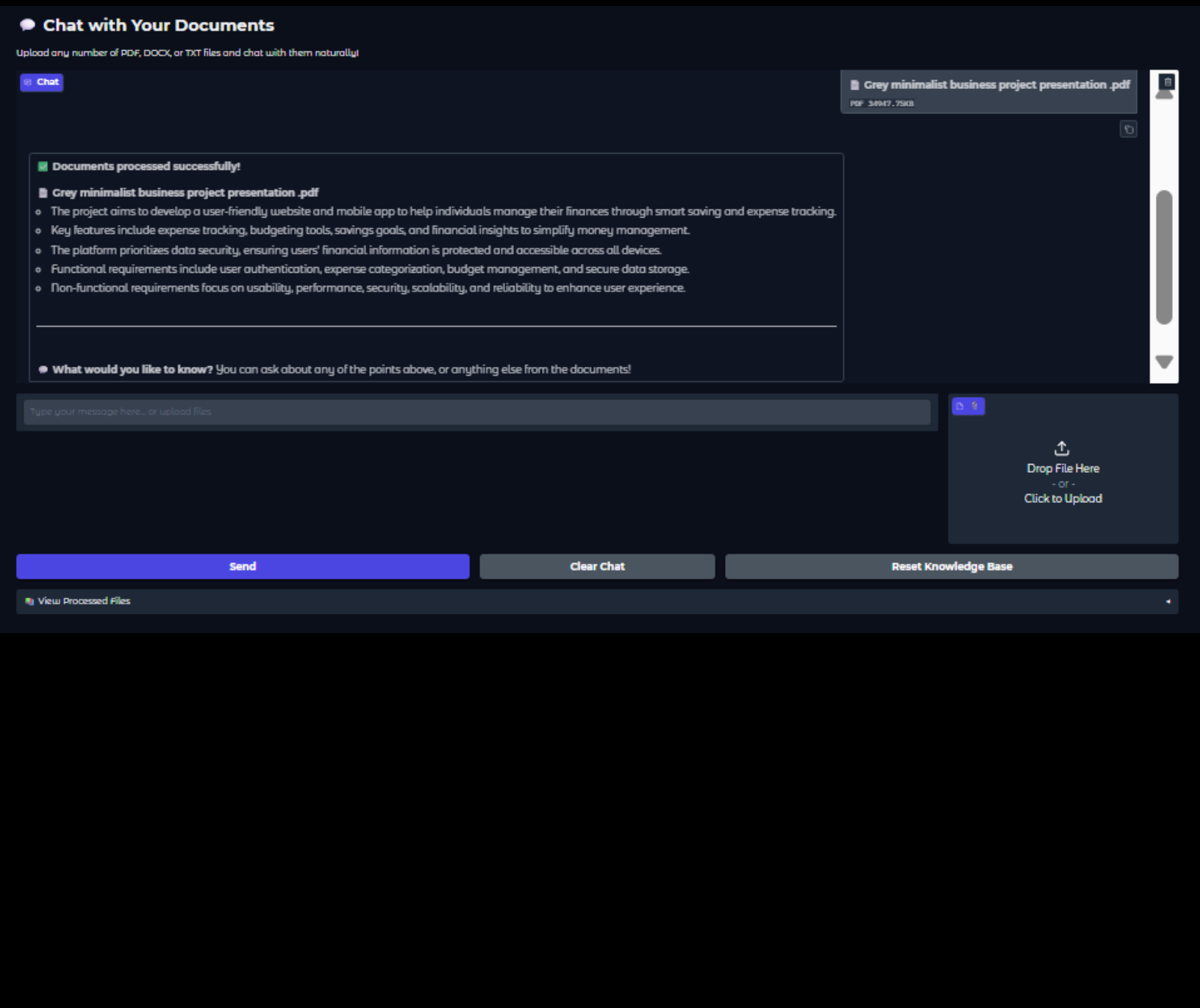
DocuChat AI


Chat with Your Documents
Upload any number of PDF, DOCX, or TXT files and chat with them naturally!
Chat
Grey minimalist business project presentation .pdf
PDF 34947.75KB
Documents processed successfully!
Grey minimalist business project presentation .pdf
The project aims to develop a user-friendly website and mobile app to help individuals manage their finances through smart saving and expense tracking.
Key features include expense tracking, budgeting tools, savings goals, and financial insights to simplify money management.
The platform prioritizes data security, ensuring users' financial information is protected and accessible across all devices.
Functional requirements include user authentication, expense categorization, budget management, and secure data storage.
Non-functional requirements focus on usability, performance, security, scalability, and reliability to enhance user experience.
What would you like to know? You can ask about any of the points above, or anything else from the documents!
Type your message here... or upload files
Drop File Here
- or -
Click to Upload
Send
Clear Chat
Reset Knowledge Base
View Processed Files

vzqyoouej

October 23, 2025

0.1 Import libraries and load API keys.

[1]:
```
import gradio as gr
import os
import time
from langchain_community.document_loaders import PyPDFLoader,␣
 ↪UnstructuredWordDocumentLoader, TextLoader
from langchain.text_splitter import CharacterTextSplitter
from langchain_community.vectorstores import Chroma
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain.chains import ConversationalRetrievalChain
from langchain.memory import ConversationBufferMemory
from dotenv import load_dotenv

# Load environment variables
load_dotenv(override=True)
os.environ['OPENAI_API_KEY'] = os.getenv('GPT_KEY', 'your-key-if-not-using-env')
```

0.2 Set AI model and document processing settings.

[2]:
```
# Model configuration
MODEL = "gpt-4o-mini"

# Advanced retrieval configuration
CHUNK_SIZE = 1000
CHUNK_OVERLAP = 200
TOP_K_RETRIEVAL = 5  # Number of relevant chunks to retrieve
SIMILARITY_THRESHOLD = 0.7  # Minimum similarity score
```

0.3 Initialize embeddings, memory, and tracking variables.

[3]:
```
# Initialize embeddings and memory
embeddings = OpenAIEmbeddings(openai_api_key=os.environ['OPENAI_API_KEY'])
memory = ConversationBufferMemory(memory_key="chat_history",␣
 ↪return_messages=True)

# Global variables
```

```
vectorstore = None
conversation_chain = None
processed_files = []
all_chunks = []
```

```
C:\Users\dell\AppData\Local\Temp\ipykernel_9944\4095068184.py:3:
LangChainDeprecationWarning: Please see the migration guide at:
https://python.langchain.com/docs/versions/migrating_memory/
  memory = ConversationBufferMemory(memory_key="chat_history",
return_messages=True)
```

0.4 Load PDF, DOCX, or TXT files based on file type.

[4]:
```
def load_document(file_path):
    """Load document based on file extension"""
    ext = file_path.split('.')[-1].lower()

    if ext == "pdf":
        loader = PyPDFLoader(file_path)
    elif ext == "docx":
        loader = UnstructuredWordDocumentLoader(file_path)
    elif ext == "txt":
        loader = TextLoader(file_path, encoding="utf-8")
    else:
        return None, f"Unsupported file type: {ext}"

    try:
        docs = loader.load()
        return docs, None
    except Exception as e:
        return None, f"Error loading document: {str(e)}"
```

0.5 Generate bullet-point summaries using GPT with streaming output.

[5]:
```
def generate_smart_summary_streaming(documents, filename):
    """Generate a simple, clean summary with key points only - with natural␣
 ↪streaming"""
    try:
        # Combine document content (limit to avoid token overflow)
        combined_text = "\n".join([doc.page_content for doc in documents[:10]])␣
 ↪ # First 10 pages/chunks
        if len(combined_text) > 8000:  # Limit text length
            combined_text = combined_text[:8000]

        llm_stream = ChatOpenAI(
            temperature=0.3,  # Lower temperature for more focused summaries
            model_name=MODEL,
```

```
            openai_api_key=os.environ['OPENAI_API_KEY'],
            streaming=True
        )

        prompt = f"""
**Role:** You are an expert document analyst specializing in creating concise, executive-level summaries.

**Objective:** Distill the core essence of the following document into its most critical takeaways. The summary should be immediately scannable and require no further explanation.

---

**Document Context:**
- **Filename:** {filename}
- **Content Snippet:**
{combined_text}

---

**Task & Strict Rules:**
1.  **Analyze** the provided content and identify the 3-5 most vital points.
2.  **Format** the output as a bulleted list. Each point must start with a hyphen (`-`).
3.  **Be Concise:** Each bullet point must be a single, impactful sentence.
4.  **No Extra Text:** Your response MUST NOT include any headers, titles, introductions, or concluding remarks. The entire output should consist only of the bullet points.

**Example of a perfect output:**
- Key insight or main finding is stated clearly and directly.
- Another crucial point or outcome is summarized in one sentence.
- The most significant conclusion or call-to-action is highlighted.

---

Generate the summary now based on the document context provided.
"""

        # Stream the summary with natural pacing
        summary_header = f"  **{filename}**\n"
        full_summary = summary_header
        yield full_summary
        time.sleep(0.05)

        for chunk in llm_stream.stream(prompt):
```

```
            if hasattr(chunk, 'content'):
                full_summary += chunk.content
                yield full_summary
                time.sleep(0.02)  # Small delay for natural reading pace

        yield full_summary + "\n"

    except Exception as e:
        yield f"  Could not generate summary for {filename}: {str(e)}"
```

0.6 Process files, generate summaries, chunk text, and build vector store.

[6]:
```
def process_multiple_documents_streaming(file_paths, chat_history):
    """Process multiple uploaded documents with streaming summaries"""
    global vectorstore, conversation_chain, processed_files, all_chunks

    new_chunks = []
    successfully_processed = []
    errors = []

    # Start with processing message
    processing_msg = "  **Documents processed successfully!**\n\n"
    chat_history.append((None, processing_msg))
    yield chat_history

    # Process each file
    for file_path in file_paths:
        filename = os.path.basename(file_path)

        # Skip if already processed
        if filename in processed_files:
            continue

        docs, err = load_document(file_path)
        if err:
            errors.append(f"  {filename}: {err}")
            continue

        # Stream the summary generation
        current_summary = ""
        for summary_chunk in generate_smart_summary_streaming(docs, filename):
            current_summary = summary_chunk
            # Update the chat with streaming summary
            all_summaries = processing_msg
            for prev_file in successfully_processed:
                all_summaries += f"  **{prev_file}**\n[Already shown]\n\n"
            all_summaries += current_summary
```

```
            chat_history[-1] = (None, all_summaries)
            yield chat_history

        # Split documents into chunks
        splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
        chunks = splitter.split_documents(docs)
        new_chunks.extend(chunks)
        successfully_processed.append(filename)
        processed_files.append(filename)

    # If we have new chunks, update or create vectorstore
    if new_chunks:
        all_chunks.extend(new_chunks)

        # Create or update vectorstore with all chunks
        if vectorstore is None:
            vectorstore = Chroma.from_documents(all_chunks, embeddings)
        else:
            # Add new documents to existing vectorstore
            vectorstore.add_documents(new_chunks)

        # Create/update conversational chain
        llm = ChatOpenAI(
            temperature=0.7,
            model_name=MODEL,
            openai_api_key=os.environ['OPENAI_API_KEY']
        )
        conversation_chain = ConversationalRetrievalChain.from_llm(
            llm,
            retriever=vectorstore.as_retriever(),
            memory=memory
        )

    # Final message with prompt
    final_msg = chat_history[-1][1]
    final_msg += "\n---\n\n  **What would you like to know?** You can ask about␣
↪any of the points above, or anything else from the documents!"

    if errors:
        final_msg += "\n\n  **Errors:**\n" + "\n".join(errors)

    chat_history[-1] = (None, final_msg)
    yield chat_history
```

0.7 Handle user messages and generate streaming responses using RAG or direct chat.

[7]:
```
def respond(message, chat_history, files):
    """Handle chat responses with optional file upload - with streaming␣
↪support"""
    global conversation_chain

    # Check if files are uploaded
    if files:
        file_paths = [file.name if hasattr(file, 'name') else file for file in␣
↪files]

        # Display uploaded files in chat first (user message side)
        files_display = "  **Uploaded Files:**\n\n"
        for file_path in file_paths:
            filename = os.path.basename(file_path)
            file_size = os.path.getsize(file_path) / 1024  # Size in KB
            file_ext = filename.split('.')[-1].upper()
            files_display += f"  **{filename}**\n`{file_ext} {file_size:.
↪2f}KB`\n\n"

        # Show files as user message
        if message and message.strip():
            # If there's also a text message, combine them
            chat_history.append((files_display + "\n" + message, ""))
        else:
            # Only files, no text message
            chat_history.append((files_display.strip(), ""))

        yield "", chat_history

        # Stream the document processing
        for updated_history in process_multiple_documents_streaming(file_paths,␣
↪chat_history):
            yield "", updated_history

        # If user also sent a message with the files, answer it
        if message and message.strip():
            # Add new user message for the question
            chat_history.append((message, ""))
            yield "", chat_history

            # Now answer their question with the newly uploaded document
            if conversation_chain:
                try:
                    llm_stream = ChatOpenAI(
```

```
                        temperature=0.7,
                        model_name=MODEL,
                        openai_api_key=os.environ['OPENAI_API_KEY'],
                        streaming=True
                    )

                    # Get relevant context
                    retriever = vectorstore.as_retriever()
                    docs = retriever.get_relevant_documents(message)
                    context = "\n\n".join([doc.page_content for doc in docs[:
 ↪3]])

                    prompt = f"""Based on the following context from the␣
 ↪documents, answer the user's question.

Context from documents:
{context}

User question: {message}

Provide a helpful, accurate answer based on the context above."""

                    full_response = ""
                    for chunk in llm_stream.stream(prompt):
                        if hasattr(chunk, 'content'):
                            full_response += chunk.content
                            # Update the last message in chat history
                            chat_history[-1] = (message, full_response)
                            yield "", chat_history
                            time.sleep(0.02)  # Natural reading pace

                except Exception as e:
                    chat_history[-1] = (message, f" Error: {str(e)}")
                    yield "", chat_history

        return

    # Handle the user's message with streaming (no files uploaded)
    if message and message.strip():
        # Add user message immediately
        chat_history.append((message, ""))

        if conversation_chain:
            # Use RAG chain for document-based questions with streaming
            try:
                llm_stream = ChatOpenAI(
                    temperature=0.7,
```

```
                    model_name=MODEL,
                    openai_api_key=os.environ['OPENAI_API_KEY'],
                    streaming=True
                )

                # Get relevant context
                retriever = vectorstore.as_retriever()
                docs = retriever.get_relevant_documents(message)
                context = "\n\n".join([doc.page_content for doc in docs[:3]])

                # Get chat history for context
                history_text = ""
                if len(chat_history) > 1:
                    for h in chat_history[-6:-1]:  # Last 5 exchanges
                        if h[0] and h[1]:
                            history_text += f"Human: {h[0]}\nAssistant:␣
↪{h[1]}\n\n"

                prompt = f"""Based on the following context from the documents,␣
↪answer the user's question.

Previous conversation:
{history_text}

Context from documents:
{context}

User question: {message}

Provide a helpful, accurate answer based on the context above."""

                full_response = ""
                for chunk in llm_stream.stream(prompt):
                    if hasattr(chunk, 'content'):
                        full_response += chunk.content
                        # Update the last message in chat history
                        chat_history[-1] = (message, full_response)
                        yield "", chat_history
                        time.sleep(0.02)  # Natural reading pace

            except Exception as e:
                chat_history[-1] = (message, f" Error: {str(e)}")
                yield "", chat_history
        else:
            # Regular chat without document context - with streaming
            try:
                llm_stream = ChatOpenAI(
```

```
                    temperature=0.7,
                    model_name=MODEL,
                    openai_api_key=os.environ['OPENAI_API_KEY'],
                    streaming=True
                )

                full_response = ""
                for chunk in llm_stream.stream(message):
                    if hasattr(chunk, 'content'):
                        full_response += chunk.content
                        # Update the last message in chat history
                        chat_history[-1] = (message, full_response)
                        yield "", chat_history
                        time.sleep(0.02)  # Natural reading pace

            except Exception as e:
                chat_history[-1] = (message, f"  Error: {str(e)}")
                yield "", chat_historyb
```

0.8 Helper functions for displaying files and resetting the system.

0.8.1 Show list of all processed documents.

[8]:
```
def show_processed_files():
    """Return list of processed files"""
    if not processed_files:
        return "No files processed yet."
    return "  Processed files:\n" + "\n".join([f"• {f}" for f in␣
 ↪processed_files])
```

0.8.2 Clear all documents and reset to initial state.

[9]:
```
def reset_knowledge_base():
    """Clear all processed documents and reset the system"""
    global vectorstore, conversation_chain, processed_files, all_chunks
    vectorstore = None
    conversation_chain = None
    processed_files = []
    all_chunks = []
    return "  Knowledge base reset. All documents cleared."
```

0.9 Create web interface with chat, file upload, and buttons.

0.9.1 Define chat window, input fields, and control buttons and Connect buttons to functions for interactivity.

[10]:
```
# Create Gradio interface
with gr.Blocks(title="Chat with Documents", theme=gr.themes.Soft()) as demo:
    gr.Markdown("#   Chat with Your Documents")
    gr.Markdown("Upload any number of PDF, DOCX, or TXT files and chat with␣
 ↪them naturally!")

    chatbot = gr.Chatbot(
        label="Chat",
        height=500,
        show_copy_button=True,
        type="tuples"
    )

    with gr.Row():
        msg = gr.Textbox(
            label="Message",
            placeholder="Type your message here... or upload files",
            scale=4,
            show_label=False
        )
        file_upload = gr.Files(
            label=" ",
            file_types=['.pdf', '.docx', '.txt'],
            file_count="multiple",
            scale=1
        )

    with gr.Row():
        submit_btn = gr.Button("Send", variant="primary", scale=2)
        clear_chat_btn = gr.Button("Clear Chat", scale=1)
        reset_btn = gr.Button("Reset Knowledge Base", variant="stop", scale=2)

    with gr.Accordion("  View Processed Files", open=False):
        files_display = gr.Textbox(label="Processed Documents",␣
 ↪interactive=False, lines=5)
        refresh_btn = gr.Button("Refresh List", size="sm")

    # Event handlers with streaming
    submit_btn.click(
        respond,
        inputs=[msg, chatbot, file_upload],
        outputs=[msg, chatbot]
    )
```

```
    msg.submit(
        respond,
        inputs=[msg, chatbot, file_upload],
        outputs=[msg, chatbot]
    )

    # Clear uploaded files after submission
    submit_btn.click(lambda: None, None, file_upload)
    msg.submit(lambda: None, None, file_upload)

    # Clear chat only (keeps documents)
    clear_chat_btn.click(lambda: [], None, chatbot)
    clear_chat_btn.click(lambda: "", None, msg)

    # Reset knowledge base
    def reset_and_notify(chat_history):
        message = reset_knowledge_base()
        chat_history.append((None, message))
        return chat_history

    reset_btn.click(reset_and_notify, inputs=[chatbot], outputs=[chatbot])

    # Refresh files list
    refresh_btn.click(show_processed_files, outputs=files_display)
    demo.load(show_processed_files, outputs=files_display)
```

```
C:\Users\dell\AppData\Local\Temp\ipykernel_9944\4146367983.py:6: UserWarning:
The 'tuples' format for chatbot messages is deprecated and will be removed in a
future version of Gradio. Please set type='messages' instead, which uses openai-
style 'role' and 'content' keys.
  chatbot = gr.Chatbot(
```

0.9.2 Start the Gradio web application.

[1]:
```
# Launch the app
demo.launch(inbrowser=True)
```